



COURSE MATERIAL

Memory and Bits

CMSC 131 Bootcamp Block 7

Prepared by: Rene Andre Bedonia Jocsing

Published: September 16, 2026

Deadline: September 16, 2026

Prerequisites

One archive holds everything this block needs, including a project that already builds. You can start today without Git, without a GitHub account, and without last session's folder.

You need this [archive](#). Unzip it somewhere permanent and work inside the folder it makes.

Session Objectives

- Choose between the `.data` and `.bss` sections for a given piece of storage
- Distinguish a label's address from the value stored at it
- Reserve and access bytes, words, and doublewords
- Compute an address with an addressing mode rather than naming one
- Save and restore a register with the stack
- Extract and set individual bits with shifts and masks

Scoring

This block is worth 10 points for work completed during its scheduled laboratory session. Your instructor checks your progress before the session ends and prorates the 10 points according to how much of the block you completed. Complete all seven guided blocks, `b1` through `b7`, without an absence and you earn a 30-point completion bonus. Attendance is checked during every bootcamp session, so it doesn't carry a separate score.

Before You Start

The archive is a working project with today's files already in it. If you prefer to keep working in your own folder, copy these across instead.

File	What it's for
<code>b7_starter.asm</code>	Today's exercise, with the prompts written and the bit work left to you
<code>pack.input</code>	Three values, fed to your program by <code>make check</code>
<code>pack.expected</code>	What a correct program prints for them
<code>b7_validation.py</code>	Runs your program against seven cases the fixture never reaches
<code>b6_solution.asm</code>	Block 6's compound interest program, finished, if you missed that session

Ninety minutes, roughly fifteen on the two sections, fifteen on labels and addresses, ten on addressing modes, ten on the stack, and forty on shifting, masking, and the exercise.

Part 1: Two Places to Put Things

Block 2 left you with six usable registers. Anything that doesn't fit lives in memory. Your program declares memory in one of two sections.

The `.data` section holds values you specify up front. They occupy space in the executable file itself.

```
1 segment .data
2 count      dd 0           ; one doubleword, initialised to 0
3 rate       dd 5
4 message    db "Balance: ", 0 ; bytes, ending in a zero
5 table      dd 10, 20, 30, 40 ; four doublewords in a row
```

The `.bss` section holds space you want reserved but not filled. Its bytes are left out of the executable file. The file records only how many bytes to reserve. The loader hands you that many bytes, zeroed, when the program starts.

```
1 segment .bss
2 buffer    resb 256         ; reserve 256 bytes
3 values    resd 100         ; reserve 100 doublewords
4 result    resd 1           ; reserve one doubleword
```

NASM calls these *sections*. The directive that opens one is spelled `segment` in this course's code, and NASM accepts `segment` and `section` as the same directive, so `section .data` means exactly what `segment .data` means.

The declaration suffixes follow a pattern:

Size	Bytes	Define with a value	Reserve space
Byte	1	db	resb
Word	2	dw	resw
Doubleword	4	dd	resd

Why does the split exist? Put a 1 MB buffer in `.data` initialised to zeroes and your executable grows by a megabyte of zero bytes, all of which has to be read off disk before the program runs. Put it in `.bss` instead and the file grows by a size field of a few bytes. The memory itself still exists once the program is running, in both cases. Only the bytes in the file differ.

The rule follows directly. If you care about the starting contents, use `.data`. If you're going to write to it before you read it, use `.bss`.

Part 2: A Label Is an Address

This is the distinction that causes the most confusion in this block. Block 2 already touched it with `print_string`.

A label denotes a **location**. Using it bare gives you the address, which is a constant the assembler works out and writes into the instruction. Wrapping it in brackets gives you the contents.

```
1 mov     eax, count          ; eax = the ADDRESS of count
2 mov     eax, [count]        ; eax = the VALUE stored at count
```

Writing works the same way:

```

1      mov     [count], eax           ; store eax into count
2      mov     dword [count], 7      ; store the literal 7 into count

```

That `dword` is required. Given `mov [count], 7`, the assembler can't tell whether you mean to write one byte, two, or four, so it refuses. Writing the size resolves it. Omitting it entirely gets you error: operation size not specified.

The brackets matter on the left side too. `mov count, eax` doesn't assemble. A bare `count` is an address. An address is a constant, no different in kind from the 7 above. You can't store into a constant any more than you can store into the number 7. `mov [count], eax` stores into the memory the address refers to, and memory is what holds values.

The address-versus-value trap from Block 2 shows up here too, now with a reason for why. `mov eax, message` loads the address. That's what `print_string` wants. `mov eax, [message]` loads the first four characters of the text as if they were a number, then prints a large meaningless integer.

If you've written C, you've met this split before. The brackets are the same idea as the unary `*` in C, the dereference operator. In C, `p` holds a pointer and `*p` is the value it points at. In assembly, a label is that pointer, and `[label]` is `*p`. The parallel runs the other way too. In C, `&x` takes the address of a variable. In assembly, a bare label is the address of a location. You never write an ampersand because the label already is one. C makes you say which side of the split you mean with `*` and `&`. Assembly says it with brackets. Both forms assemble, so a wrong guess is a wrong number and the program runs anyway.

Part 3: Addressing Modes

Defining a label gets you one fixed location. Data in the wild comes in runs. You need the address computed as you go.

```

1      mov     eax, [table]           ; the first element
2      mov     eax, [table + 4]       ; the second, since a dd is 4 bytes
3      mov     eax, [table + ebx]     ; ebx bytes past the start
4      mov     eax, [table + ebx*4]   ; element number ebx
5      mov     eax, [ebx + esi*4 + 8] ; the general form

```

The last two are the useful ones. That `*4` is a **scale**. It exists because indexing an array of double-words means multiplying the index by four. The multiply isn't a separate instruction. The processor works out `base + index*scale + offset` as one effective address, so `[table + ebx*4]` is a single memory operand.

So walking an array is:

```

1      mov     ebx, 0                 ; index
2      sum_loop:
3      cmp     ebx, 4
4      jge     sum_done
5      mov     eax, [table + ebx*4]   ; element ebx
6      add     esi, eax
7      inc     ebx                    ; next index, not next byte
8      jmp     sum_loop
9      sum_done:

```

Note `inc ebx` advances by one *element*. The `*4` handles the byte arithmetic. Writing `add ebx, 4` here and keeping the scale would step four elements at a time.

The scale may only be 1, 2, 4, or 8. Those are the sizes the hardware supports.

Part 4: The Stack

There's one more place to put things. You've been using it since Block 2 without knowing its name.

Block 2 said `esp` and `ebp` were spoken for. This is what they're spoken for by. The **stack** is a region of memory that grows downward from high addresses. `esp`, the stack pointer, always holds the address of the item on top of it. You don't index into it the way you index into `table`. You push things on and pop them off.

```
1      push    eax                ; put eax on top of the stack
2      pop     eax                ; take the top item back off, into eax
```

`push` subtracts four from `esp` and writes there. `pop` reads and adds four back. Nothing else is going on.

What it gives you is a place to put a value when you've run out of registers, which by now must have happened to you at least once:

```
1      push    eax                ; the answer, which print_string is about to destroy
2      mov     eax, message
3      call    print_string
4      pop     eax                ; and it's back
5      call    print_int
```

Follow two values through it. Say `eax` holds 7 and `ebx` holds 9, and `esp` is 0x1000:

```
1      push    eax      esp = 0x0FFC   [0x0FFC] = 7
2      push    ebx      esp = 0x0FF8   [0x0FF8] = 9      <- esp points here now
3      pop     ebx      ebx = 9        esp = 0x0FFC
4      pop     eax      eax = 7        esp = 0x1000
```

The first `pop` reads whatever `esp` points at. That's the 9, because the 9 went on last. So the 9 comes off first, and the 7, which went on first, comes off last. Swap the two pops and `eax` gets 9 while `ebx` gets 7. Nothing reports it. The value pushed last is the value popped first. That rule is called last in, first out, and it's why this storage is called a *stack*. A stack of plates comes apart the same way.

What's important at the end is that `esp` is back where it started before the `ret`. One `pop` for each `push` is the simplest way to get there. Two pushes and two pops that cancel out is what your routine owes its caller. Run the `ret` with `esp` four bytes off and the address it jumps to is whatever number happens to sit there.

The Two Lines You've Been Copying Since Block 2

```
1      pusha
2      popa
```

`pusha` pushes all eight general-purpose registers, in the order `eax`, `ecx`, `edx`, `ebx`, `esp`, `ebp`, `esi`, `edi`. The fifth slot holds `esp` from before the instruction started.

`popa` pops them back in the reverse order, with one exception. It reads the saved `esp` slot and throws it away. Loading the old `esp` would undo the pops that had already happened. That's why the `pusha/pop` pair leaves `esp` exactly where it found it.

Their job is a promise to whoever called the subprogram. `driver.c` called your `asm_main` with registers it may still care about. Those two lines hand every one of them back untouched. It's the same

promise Carter's routines make to you. That's why the value you left in `esi` in Block 2 was still there after a call `print_string`. Now you know how they kept it.

Look at the end of every program you've written and you'll see one more line after the `popa`:

```

1      popa
2      mov     eax, 0
3      leave
4      ret

```

The `mov` isn't tidying up. `_asm_main` returns an `int` to `driver.c` in `eax`, and `driver.c` hands that number to the operating system as the exit status. `popa` has just put the original `eax` back, so the `mov` after it sets the value `main` returns. Move it before the `popa` and the exit status becomes whatever the caller had in `eax`.

`enter 0,0` and `leave` are a matched pair in the same spirit, setting up and tearing down `ebp` as a fixed reference point for the duration of your routine. Block 8 leans on none of this beyond what you've already been copying, but Laboratory Activity 2 is built on top of it.

Part 5: Shifting and Masking

Everything so far has treated a register as a number. Sometimes it's a collection of separate fields packed together. What if you need one field out of the middle? Laboratory Activity 1 is built entirely on this, so here's the groundwork.

Shifting

```

1      shl     eax, 3           ; shift left 3 places, filling with zeroes
2      shr     eax, 3           ; shift right 3, filling with zeroes
3      sar     eax, 3           ; shift right 3, filling with the sign bit

```

`shl` moves every bit up by the shift count and fills the low end with zeroes. Call the count n . The result is the value multiplied by 2 to the power n , so a shift by 3 multiplies by 8, as long as nothing leaves the top. Bits that do leave the top are gone, so the instruction keeps only the part of the product that fits the operand's width.

`shr` moves every bit down and fills the top with zeroes. Read it unsigned, it divides by 2 to the power n and rounds down. Whatever falls off the low end is discarded, the same way `div` discards a remainder.

The count has a limit that isn't the one you'd guess. A 32-bit x86 shift keeps only the low five bits of its count, so the count is always 0 through 31. Shift a 32-bit register by 32 and the processor shifts it by 0, because 32 is 100000 in binary and the low five bits of that are zero. Shift by 33 and it shifts by 1. That's a trap for a loop that wants to clear a register with one big shift. Every shift in this block, and in Laboratory Activity 1, uses a count from 1 to 31, where the count means what it says.

`sar` also moves bits down and discards what falls off, but it fills the top with a copy of the sign bit. Read the value signed, the result is the same division rounded *down*, which for a negative number means away from zero. -5 shifted right by one is -3, not -2. The discarded bits round the result toward negative infinity.

Watch the bits move. The example is an eight-bit operand, `al`:

```

1      al = 0110 0101
2           ||
3      0001 1001 01      the two low bits fall off, zeroes in at the left

```

`al` held 0x65, which is 101. Shifted right by two the bits become 0001 1001, or 0x19, which is 25. That's 101 divided by four, with the remainder discarded.

Shifting left works the same way in the other direction:

```

1  al = 0110 0101
2      ||
3  1001 0100          zeroes in at the right, the top bits are gone

```

That's 0x94 here, which looks like a negative number if you read it signed. Multiplying by four can overflow into the sign bit, and nothing reports it. The bits that leave the top end are gone the same way the ones below were.

shr and sar differ only for negative numbers. shr pulls in zeroes, so a negative number becomes a large positive one. sar replicates the sign bit, preserving the sign. The same signed-versus-unsigned split as Block 3 and Block 5, in a third place.

```

1  eax = 0x80000000    the most negative 32-bit number
2  shr  eax, 1         -> 0x40000000, a large positive number
3  sar  eax, 1         -> 0xC0000000, still negative, now -1073741824

```

A shift by a count of one or more also writes the last bit it pushed out into the carry flag. That bit is what you inspect when you shift one bit at a time. Shift by more than one and only the final bit survives. Each step overwrites the flag. The earlier bits are already gone.

Masking

```

1  and    eax, 0x0F      ; keep only the low 4 bits, clear the rest
2  or     eax, 0x80      ; force bit 7 on, leave others alone
3  xor    eax, 0xFF      ; flip the low 8 bits
4  not    eax            ; flip all 32

```

and clears. or sets. xor flips. A **mask** is just a constant with ones where you want to act.

Getting a Field Out

The two combine into one standard move. To read a field, shift it down to the bottom, then mask it to its width.

Take a byte holding two 4-bit fields, version on top and length beneath:

```

1  0100 0101
2  ---- ----
3  ver  len

```

```

1  mov     eax, 0x45
2
3  mov     ebx, eax
4  shr     ebx, 4          ; ebx = 0x04, the version
5  and     ebx, 0x0F       ; keep four bits, clear anything above
6
7  mov     ecx, eax
8  and     ecx, 0x0F       ; ecx = 0x05, the length. No shift needed.

```

Follow the same byte through that code, one step at a time. 0x45 is 0100 0101. The version is the top four bits, 0100. The length is the bottom four, 0101.

```

1  starting value    0100 0101
2  shr  ebx, 4      0000 0100  0101    the four low bits fall off
3                                     ~~~~    all four are discarded
4  and  ebx, 0x0F   0000 0100    bits 4 and up are already zero

```

The `shr` moved the version down to bit 0 and pushed all four bits of the length off the bottom. With a wider register, the same shift would bring down whatever sat above the field as well. The `and` clears those high bits, so it stays.

For the length, the `and` does the whole job because the field already sits at the bottom:

```

1  starting value    0100 0101
2  and  ecx, 0x0F   0000 0101    the version clears out: length 5

```

Shift first, then mask with a low mask whose width is the field's width. Write it that way in your head as `(value >> start) & low_mask`. It works for any field, wherever it sits.

A second correct form exists. Mask the field in place with a mask built at its position, then shift it down:

```

1      mov     ecx, eax
2      and     ecx, 0xF0      ; keep bits 4 to 7 where they are
3      shr     ecx, 4        ; then move them down

```

That's `(value & positioned_mask) >> start`. It gives the same answer for the same field. Both orders work as long as the mask matches the field's position and width.

What fails is pairing a low mask with a shift when the field isn't at the bottom. `and eax, 0x0F` clears every bit above bit 3, so a field sitting at bit 4 or higher is gone before any shift can reach it. The mask has to describe where the field sits and its width.

This is the operation Laboratory Activity 1 asks for on a network packet header. The awkward field there straddles a byte boundary, so it needs two of these, combined. Getting comfy now is worth the discomfort.

Part 6: Exercise

Start from `b7_starter.asm`, which reads the three values already and leaves the bit work to you.

```

1  cp b7_starter.asm pack.asm

```

Write a program that packs and unpacks a small record.

1. Reserve one doubleword in `.bss` called `packed`.
2. Read three values from the user: a version (0 to 15), a flag (0 or 1), and a length (0 to 255).
3. Mask each value to its field width before packing it. A version of 20 becomes 4, a flag of 2 becomes 0, and a length of 325 becomes 69.
4. Pack the three fields into a single doubleword with version in bits 12 to 15, the flag in bit 8, and length in bits 0 to 7. Store it in `packed`.
5. Print the packed value.
6. Then read it back from memory and extract all three fields again, printing each.
7. Confirm the extracted values match what was entered, after masking.

Expected output:

```

1 Version (0-15): 4
2 Flag (0 or 1): 1
3 Length (0-255): 69
4
5 Packed: 16709
6 Unpacked version: 4
7 Unpacked flag: 1
8 Unpacked length: 69

```

Check the packed value by hand. Version 4 shifted left 12 is 16384. The flag shifted left 8 is 256. The length is 69. Added together that's 16709, which matches.

Two more cases to check by hand before you run them:

At the top of every field: version 15, flag 1, length 255. Version 15 shifted left 12 is 61440, the flag shifted left 8 is 256, and the length is 255. The three add to **61951**, the largest value these three fields can produce. It isn't the largest 16-bit number, because bits 9 through 11 sit between the flag and the version and this layout leaves them zero.

Over the width of a field, an unmasked input goes somewhere it doesn't belong. Version 20 is 10100, five bits wide. Shifted left twelve places, its top bit lands at bit 16. This layout leaves bit 16 unused, so an unmasked version 20 corrupts the packed value: 20 shifted left 12 is 81920, where 4 shifted left 12 is 16384. What comes back out depends on where you mask. Unpack with a shift by 12 and a mask of 0x0F, and bit 16 is cleared on the way out, so the program prints version 4 from a packed value that was wrong all along. Unpack with the shift alone and the program prints version 20. The mask on the way in is what makes the packed value right. The mask on the way out is what makes the unpacked field right. The exercise asks for both.

Length 325 is worse. It's 1 0100 0101, nine bits wide. Its low eight bits are 69. Its ninth bit is 1. Packed unmasked, that ninth bit lands at bit 8, which is the flag. An unmasked length of 325 sets the flag. The field you wrote two lines apart from it quietly changes. Mask every input to its field width. The three fields then stay in their own places. Then (20, 2, 325) enters as (4, 0, 69) and unpacks as (4, 0, 69).

Which instruction from Part 5 clears the bits you don't want, and where would it have to go to help? You know the answer already.

Checking it

```

1 make PROG=pack check

```

pack.input holds 4, 1, and 69, one per line. That's the one case the fixture covers. For the rest:

```

1 python b7_validation.py

```

On Linux that's python3. It types seven cases at your program: the sample, zeroes, the maximum, each field on its own, and one input wider than two of its fields at once. It reads only the four report lines, from Packed: onward, so your prompts are yours to word. Each case is its own run. A run has to exit cleanly and print nothing to stderr.

One thing the script can't see: whether the packed value went through memory at all. A program that keeps it in a register prints the same four lines. Your instructor checks the round trip at your desk, by reading the code and by watching the store and the loads in the debugger. The exercise asks for the round trip. The checker can't, so a person does.

Build first. The validator runs the program and expects it to exist:

```
1 make PROG=pack
```

Testing Checklist

Core Functionality

- Version 4, flag 1, length 69 packs to 16709
- Every unpacked field matches what was entered
- Version 0, flag 0, length 0 packs to 0 and unpacks to zeroes
- Version 15, flag 1, length 255 packs to 61951, the largest the three fields produce
- An over-width input is masked to its field, so (20, 2, 325) unpacks as (4, 0, 69)
- The packed value survives being stored to .bss and read back
- `make PROG=pack check` prints OK: pack matches pack.expected

Common Pitfalls

- Using `mov eax, packed` when you meant `mov eax, [packed]`
- Omitting `dword` on a store of a literal
- Pairing a low mask with a shift, which clears any field that sits above the mask
- Leaving an over-width input unmasked, so its extra bits land outside the field
- Using `add ebx, 4` alongside a `*4` scale, stepping four elements at a time
- A scale other than 1, 2, 4, or 8, which won't assemble

Architecture Review

What We Built

- Storage in both sections, chosen for the right reason
- Array traversal with a scaled index rather than manual byte arithmetic
- A place to put a value when the registers run out
- A packed record built with shifts and masks, and taken apart again

Key Takeaways

1. **.data for contents you care about, .bss for space you'll fill.** The second keeps its bytes out of the file. The memory still exists at run time.
2. **A bare label is an address, and brackets read the value.** This one distinction explains most early memory bugs.
3. **A scaled index costs no extra multiply.** The processor computes `base + index*scale + offset` as one effective address.
4. **Balance the stack before the `ret`.** In straight-line code that means one `pop` per `push`, in the opposite order.
5. **`pusha` and `popa` were a promise all along**, which is why your registers survived Carter's routines. The `mov eax, 0` after the `popa` is what sets the exit status.
6. **and clears, or sets, xor flips.** A mask marks where to act.
7. **Shift down, then mask with a low mask.** Masking in place works too, if the mask sits at the field's position.

Next Session Preview

- A timed exercise, worked alone, covering everything from Blocks 1 through 7
- Forming groups for the rest of the semester
- Laboratory Activity 1 released at the end of the session